

SajdaTime — Requirements and Architecture

Version 1.0.0 · Android · Kotlin + Jetpack Compose

This document is the handoff record: what was built, why each decision was made, the exact business rules an iOS port must reproduce, and what deliberately was not built.

1. Product intent

SajdaTime is a charity project (sadaqah jariyah). It is free forever, carries no ads and no in-app purchases, and collects nothing.

Three constraints shaped every technical decision:

Constraint	Consequence
Zero running cost	Prayer times are computed on-device. There is no server, and no paid API.
Privacy by design	Only coarse location, only in the foreground, never transmitted, never backed up.
Solo maintainer	Minimal dependencies, no build-time codegen, no framework the author must relearn.

2. Technology choices

Area	Choice	Why
Language	Kotlin	Official, concise.
UI	Jetpack Compose + Material 3	Less code than XML, better animation, one source of truth for theming.
Min SDK	24 (Android 7.0)	Covers the overwhelming majority of active devices worldwide without carrying Android 5 baggage.
Target SDK	36	Meets Google Play's requirement. Compiled against SDK 37.
Calculation	<code>com.batoulapps.adhan:adhan:1.2.1</code> (MIT)	Fully offline, well tested, zero transitive dependencies.
Storage	DataStore Preferences	Async, no SharedPreferences main-thread traps.
Scheduling	<code>AlarmManager</code> + <code>WorkManager</code>	Exact alerts plus a daily safety net.
PDF	Android <code>PdfDocument</code>	Built into the platform. No library, no licence, no APK weight.
Networking	<code>HttpURLConnection</code> + <code>org.json</code>	One optional GET request does not justify Retrofit/OkHttp/Moshi.
Java 8+ APIs	Core library desugaring	Gives <code>java.time</code> , including the Hijri calendar, down to API 24.

Deliberately not used

- **Google Play Services location.** The platform `LocationManager` is enough for coarse accuracy, removes a proprietary dependency, and keeps the app working on de-Googled devices. Prayer times shift by well under a minute across a coarse-location cell.
- **Material You dynamic colour.** A wallpaper-derived palette would break the verified contrast guarantees and the deliberate green/gold identity.
- **Dependency injection framework.** Three repositories constructed in one ViewModel.
- **Navigation library.** Two screens and an onboarding flow. A boolean is enough.

3. Prayer calculation — the business rules

This section is the specification an iOS port must match exactly.

3.1 Method selection

`CalcMethod.AUTO` resolves at calculation time from the user's sect:

- Sunni → Muslim World League
- Shia → Jafari (Ithna Ashari)

A user who never opens advanced settings still gets a correct convention. An explicitly chosen method always overrides AUTO.

3.2 Asr (madhab)

Asr depends only on the shadow ratio:

Madhab	Shadow ratio	adhan value
Hanafi	2× object length	Madhab.HANAFI
Shafi'i, Maliki, Hanbali	1× object length	Madhab.SHAFI
Jafari	1× object length	Madhab.SHAFI

Three of the four Sunni madhabs therefore produce identical times. This is correct, not a bug, and is asserted in `DeterminismTest`.

3.3 Angles

Method	Fajr	Maghrib	Isha
Muslim World League	18°	sunset	17°
Umm al-Qura	18.5°	sunset	sunset + 90 min (120 in Ramadan)
Jafari (Ithna Ashari)	16°	4° below horizon	14°
Tehran (Geophysics)	17.7°	4.5° below horizon	14°

Other Sunni conventions use adhan's bundled parameters unchanged.

3.4 The Jafari Maghrib rule — the one non-obvious piece

For Sunni conventions, Maghrib is sunset. For Jafari and Tehran, Maghrib is when the redness leaves the eastern sky, defined as the sun sitting 4° (4.5° for Tehran) below the horizon. That is typically **15–20 minutes after sunset**, and it is the moment fasts are broken — so getting it wrong is not cosmetic.

adhan exposes no Maghrib angle. Neither `adhan-java` nor `adhan2` models one.

Rather than hand-rolling solar astronomy, the engine reuses adhan's own validated maths: adhan's Isha calculation is exactly "the evening time at N degrees of solar depression". So the engine runs a second calculation with `ishaAngle` set to the Maghrib angle and reads the resulting Isha.

```
val probe = CalculationParameters(0.0, 4.0, CalculationMethod.OTHER)
val jafariMaghrib = PrayerTimes(coords, date, probe).isha
```

This was verified to the minute against the Aladhan API's independent Shia Ithna-Ashari implementation — all six times matched exactly for Tehran on 21 June 2026, including Maghrib at 19:42 against a sunset of 19:24. See `PrayerEngineTest` .

An iOS port using adhan-swift must reproduce this, since adhan-swift has the same gap.

3.5 High latitude rule

Above roughly 48° latitude the sun never dips far enough below the horizon in summer for a true Fajr or Isha to exist. A fallback is mandatory.

The engine uses `HighLatitudeRule.TWILIGHT_ANGLE` , which divides the night in proportion to each prayer's twilight angle.

adhan's own default, `MIDDLE_OF_THE_NIGHT` , is unusable at UK latitudes: for London on 21 June it collapses Fajr and Isha onto the *same instant* (both 01:02), leaving no window for Isha at all.

`TWILIGHT_ANGLE` produces Fajr 02:31 and Isha 23:27 for the same date, matching Aladhan exactly.

This is set explicitly in `PrayerEngine` , not left to the library default.

3.6 Ramadan (Umm al-Qura only)

The official Umm al-Qura calendar stretches the Isha interval from 90 to 120 minutes during Ramadan. adhan implements only the 90-minute rule, so the engine adds the extra 30 minutes itself.

Ramadan is detected as Hijri month 9 via `java.time.chrono.HijrahDate` — and `HijrahChronology` is the Umm al-Qura calendar, so the two agree by construction. The adjustment applies to no other method.

3.7 Determinism

adhan rounds its results to the nearest minute but leaves the millisecond field carrying the wall-clock time of the call. Two identical computations therefore return instants a few milliseconds apart.

The engine truncates every result to the minute. Without this, the countdown would jitter and alarms would land at an arbitrary point within their minute. Guarded by `DeterminismTest` .

3.8 Next prayer

`nextPrayer` scans **yesterday, today and tomorrow**, not just today. At high latitudes Isha can fall after midnight and therefore belongs to the previous calendar day's timetable while still being genuinely upcoming. Scanning today alone would silently skip it. The alarm scheduler applies the same rule.

Sunrise is displayed but is never returned as "the next prayer" and is never notified.

4. Notifications and reliability

Prayer alerts are the feature most likely to fail silently, so the schedule is rebuilt from four independent triggers:

1. every time the app is opened

2. every time an alarm fires (chains the next window)
3. a `WorkManager` job every 12 hours
4. device boot, time-zone change, clock change, and app update

Alarms are laid down two days ahead, so a single missed trigger cannot break the chain.

Exact alarm strategy

Condition	Mechanism	Trade-off
<code>canScheduleExactAlarms()</code> is true	<code>setExactAndAllowWhileIdle</code>	Exact, Doze-proof, no status bar icon.
Permission refused (Android 12+)	<code>setAlarmClock</code>	Still exact and permission-free, but shows an alarm icon.

A late Fajr notification is worse than a status bar icon, so the app never silently degrades to inexact alarms. Settings shows a prompt linking to the system screen when the permission is missing.

Channels

- `prayer_times` — `IMPORTANCE_HIGH`, vibrates. One notification per prayer.
- `next_prayer_badge` — `IMPORTANCE_LOW`, silent, ongoing. Optional, off by default.

The badge refreshes when something has already woken the app rather than ticking every minute; a live-updating countdown would require a foreground service and a permanent battery cost for information already visible in the app.

5. Design system

Palette

Deep green (traditionally associated with Islam) as the primary, one warm gold accent, warm-neutral paper surfaces. Every hue carries meaning; nothing is decorative.

Role	Light	Dark
Primary	<code>#14624B</code>	<code>#7FD1AE</code>
Accent	<code>#8A5200</code>	<code>#E3B341</code>
Background	<code>#FBFAF7</code>	<code>#0E1512</code>
Surface	<code>#FFFFFF</code>	<code>#18211D</code>
On surface	<code>#12211C</code>	<code>#E8EEEE</code>
Secondary text	<code>#43524B</code>	<code>#B3C2BA</code>

Accessibility is enforced, not asserted. `ColorContrastTest` computes the WCAG 2.1 relative luminance of every foreground/background pair the UI actually uses and fails the build below 4.5:1 for text and 3:1 for icons and outlines. Editing a colour below threshold breaks the build.

Typography

The system font family, at system-scalable `sp` sizes, with nothing below 14sp. Bundling a webfont would add roughly 400KB to a charity app for no legibility gain, and would override the font a user may have chosen for accessibility reasons.

Time and countdown text uses tabular figures (`tnum`) so digits do not shift width as they change.

Other accessibility measures

- The next prayer is marked with a text label as well as a colour highlight, so the information does not depend on colour alone.
- The per-second countdown is hidden from screen readers and replaced with a readable summary ("Maghrib in 2h 14m"); announcing the digits every second would make the screen unusable with TalkBack.
- The Arabic bismillah carries an explicit content description.
- Touch targets are at least 48dp.
- Strings resolve through `stringResource` , so a language change while a screen is open re-reads correctly.

6. Privacy model

Data	Where it lives	Leaves the device?
Coordinates	<code>DataStore</code> , on device	Never
City name	<code>DataStore</code> , on device	Never
Sect, madhab, method	<code>DataStore</code> , on device	Never
Notification settings	<code>DataStore</code> , on device	Never
Typed city name (fallback only)	Sent once to Aladhan	Yes, with prior on-screen disclosure

Cloud backup is **disabled** (`allowBackup="false"`). Android's backup service would otherwise copy the cached coordinates to Google's servers, contradicting the app's own privacy promise. Re-entering settings takes two taps; the guarantee is worth more.

Permissions requested: `ACCESS_COARSE_LOCATION` , `POST_NOTIFICATIONS` , `SCHEDULE_EXACT_ALARM` , `RECEIVE_BOOT_COMPLETED` , `INTERNET` . No fine location, no background location.

7. Testing

23 unit tests, all offline and deterministic.

Suite	Covers
<code>PrayerEngineTest</code>	Reference timetables for Makkah, London and Tehran; the Jafari Maghrib rule; madhab differences; high-latitude behaviour; the Ramadan adjustment; next-prayer roll-over and midnight spillover; chronological ordering across 3 cities × 4 methods × 52 weeks.
<code>DeterminismTest</code>	Repeated computation stability, minute-boundary alignment, madhab equivalence.
<code>ColorContrastTest</code>	WCAG AA for every colour pair in both themes.

Reference values were captured from the Aladhan API — an independent implementation of the same conventions — and pinned as golden values so the suite stays offline. To regenerate:

```
curl "https://api.aladhan.com/v1/timings/DD-MM-YYYY?latitude=..&longitude=..&method=N"
```

(method `0` = Shia Ithna-Ashari, `3` = MWL, `4` = Umm al-Qura)

8. Porting to iOS

The `core/` package contains no Android imports. Everything in section 3 transfers directly.

1. Use [adhan-swift](#), whose API mirrors `adhan-java`. Translate `PrayerEngine.kt` more or less line for line.
2. **Reimplement the Jafari Maghrib probe (3.4).** `adhan-swift` has the same missing angle.
3. **Set the high-latitude rule explicitly (3.5).** The Swift default has the same problem.
4. Carry over the Ramadan adjustment (3.6) and minute truncation (3.7).
5. Replace `UserNotifications` for scheduling — iOS caps pending notifications at 64, so schedule roughly the next 12 days of prayers and top up on each app launch.
6. `UNUserNotificationCenter` cannot be woken on boot, so the "reschedule on boot" strategy does not apply; refresh on launch and on significant time change instead.
7. `PDFKit` replaces `PdfDocument`. `CoreLocation` with `kCLLocationAccuracyKilometer` replaces `LocationManager`.
8. Keep `ColorContrastTest` — port it, do not drop it.

9. Known limitations and next steps

Honest list of what is not covered:

- **Qibla compass** — not built. Out of scope for v1.

- **High-latitude rule is not user-selectable.** `TWILIGHT_ANGLE` is a good default and matches Aladhan, but some UK mosques publish one-seventh-of-the-night times. If users report a mismatch with their local mosque, exposing this setting is the first thing to add.
 - **Per-prayer manual offsets** — some communities apply a few minutes' adjustment. Not built; the data model would take it easily.
 - **No adhan audio** — notifications use the system sound.
 - **City search requires internet** and is only reachable if location is declined.
 - **No instrumented UI tests.** Logic and colour are covered by unit tests; the Compose screens are verified manually.
 - **App is unsigned.** A release keystore must be generated before publishing; the release build currently produces `app-release-unsigned.apk`.
 - **Widgets, Wear OS, tablet-optimised layouts** — none.
-

Made with love, free for the Ummah.